Morva

Product Requirements Document

Every wallet. Every chain. One way to get paid.
Built on NEAR Intents.

Owner	James Victor
Version	Draft v2
Date	23 October 2026
Target	NEAR Hackathon, then production
Status	For review

Contents

0. What changed in v2
1. Overview
2. Problem statement
3. Why now, why NEAR
4. Goals and non-goals
5. Target users and wedge market
6. Competitive landscape
7. Solution
8. Features and priorities
9. User flows
10. Order lifecycle
11. System architecture
12. NEAR integration
13. Data model
14. API surface
15. Repository and tech stack
16. Security and trust
17. Business model
18. Success metrics
19. Risks and mitigations
20. Roadmap
21. Open questions
22. Sources

0. What changed in v2

Change	Why
New positioning line	Leads with what a merchant gains on day one: reach to every crypto holder
Unified balance (pay from several chains at once)	Buyers' funds are often scattered across chains; NEAR Intents can combine them under one wallet address
Signed-intent payment path, including passkeys	Buyers with funds already in NEAR Intents pay with one signature and no on-chain deposit
Merchant balances held in NEAR Intents	Settlement becomes a ledger update; merchants withdraw to any chain when they choose
Escrow design clarified	The Verifier has no native hold primitive, so Morva's contract provides it
How NEAR Intents works (section 12)	Grounds the design in the Verifier ledger, intent types and swap types
Fee correction: 0.25%, not 0.2%	Current docs state 25 bps on unauthenticated 1Click quotes
New open questions to verify in the spike	Escrow order matching, contract deposits for another account, address mapping

1. Overview

> **Morva lets any business get paid by anyone holding crypto, from any wallet, on any chain, in the currency the business wants.**

Under the hood, Morva runs on NEAR Intents and adds what merchants need after the payment: holds, refunds, splits, reconciliation and payouts. Settlement through NEAR Intents is the foundation, not the product. Any team can wrap the 1Click API in a weekend and ship a "pay with any token" button. Morva is the commerce layer merchants need on top of that: the parts that turn a payment into a completed, refundable, reconcilable order.

Product at a glance

	Description
What	Hosted checkout, payment links, embeddable widget, merchant dashboard and API
For whom	Event organizers first (ticketing), then online merchants and developers
How it settles	NEAR Intents: buyer pays from 35 chains, or from several at once; merchant receives one asset
What's different	Unified buyer balance, escrow holds, refund to source, revenue splits, and a per-merchant ops agent
Revenue	Fee on every payment via 1Click appFees, plus a paid tier for ops and privacy

2. Problem statement

> "The demand to pay with crypto is real. The gap between holding crypto and spending it is where checkout loses people."

The numbers

Signal	Figure
Crypto owners who would prefer to pay with crypto	~65%
Share of global transaction value paid directly in crypto	0.19%
Users who drop off at the wallet-connection step	~30%
Users lost during Web3 wallet setup	~55%
Crypto payments that arrive for the wrong amount	More than 1 in 6
Typical crypto invoice payment window	10–20 minutes
Lost to crypto scams in 2025	$14B+
US merchants asked by customers about paying with crypto	88%

Six failures in today's crypto checkout

1. **Wrong network means lost money.** The buyer must pick the right chain. A mistake is a permanent loss, or a payment the merchant's system never sees.
2. **Payments fail even when the buyer does everything right.** Wallets and exchanges deduct fees in transit, so payments arrive short. Slow confirmations turn valid payments into "expired" invoices.
3. **Refunds are a maze.** Merchants and processors bounce buyers between them. Refunds come back at a different price, minus fees, and some processors won't refund small amounts at all.
4. **Holding money isn't enough to pay.** A buyer with USDC but no gas token cannot send it.
5. **Signing fear.** After years of wallet drainers, every "Connect" and "Sign" button starts with a trust deficit.
6. **Money is scattered across chains.** A buyer with $10 spread over four chains cannot pay $10 on a fifth without bridging everything first.

What NEAR Intents solves, and what it doesn't

NEAR Intents and its 1Click API solve routing: a buyer pays from almost any chain and the merchant receives the asset they chose. But 1Click is a **swap API, not a commerce system**. It has no orders or receipts, no merchant-initiated refunds, no hold-and-capture, no revenue splits and no reconciliation. Some swap types don't refund at all; failed swaps retry every five minutes instead. Every merchant still has to build the commerce layer themselves. That gap is Morva.

3. Why now, why NEAR

- **Proven settlement.** NEAR Intents has processed $30B+ in cumulative volume across 35 chains and 100+ assets, including Bitcoin and Solana.
- **Built-in revenue.** Distribution partners earn fees on every swap through the 1Click `appFees` parameter, with a default 50/50 share with the protocol.

- **Primitives other ecosystems lack.** Chain Signatures let one NEAR account sign on any chain. Shade Agents run verifiable agents in TEEs. Confidential Intents add private execution with one API parameter.
- **Public demand from NEAR leadership.** Illia Polosukhin listed a universal checkout via intents as an idea he wants built. Because that idea is public, the advantage goes to whoever builds the deeper layer, not the first widget.

4. Goals and non-goals

Goals for v1

- A buyer can pay from any supported wallet, token or chain, or from an exchange, with **one total and one confirmation**.
- A buyer whose funds are split across several supported chains can pay **from all of them in one checkout**.
- The merchant always receives the **exact invoice amount** in the asset and chain they chose.
- The merchant can **hold, capture, refund and split** payments without touching a block explorer.
- Late, short, over and failed payments resolve themselves without a support ticket.
- At least one real merchant takes live payments during the hackathon.

Non-goals for v1

- Fiat on-ramps or card payments.
- Local-currency settlement, such as naira payouts (revisit after v1).
- Buy-now-pay-later or credit.
- Running Morva's own solver or liquidity.
- Shopify App Store and WordPress.org marketplace listings (built after v1).

5. Target users and wedge market

Wedge: crypto events and ticketing

Hackathon side events, conferences, meetups and parties sell tickets to global crypto audiences holding funds on dozens of chains. They need exactly the hard parts generic checkouts skip: refunds when events are cancelled, holds until check-in, revenue splits with co-hosts and venues, and clean records. PartyStack is the first integration partner, giving Morva live volume from day one.

Expansion

Online merchants and digital-goods sellers through Shopify and WooCommerce plugins, then developers through the JS SDK, then AI agents paying merchants programmatically.

Personas

Persona	Needs	Pain today
Event organizer	Sell tickets, refund on cancellation, split revenue	Manual refunds, chasing co-hosts for payouts, spreadsheets
Online merchant	Accept crypto without extra support work	Underpayments, expired invoices, refund requests
Buyer	Pay with what they already hold, safely	Wrong network, gas, countdown timers, signing fear
Developer	Add checkout to an app quickly	Must build orders, refunds and webhooks themselves
AI agent (later)	Pay merchants programmatically	No unified merchant endpoint across chains

6. Competitive landscape

Product	What it does	Where it stops
HOT Pay	NEAR-ecosystem gateway: 50+ tokens, 20+ chains, pay links, embeds, webhooks, Telegram	Payment acceptance; no holds, splits or policy-driven refunds
near.com Universal Send	Scan a QR, pay anyone in any asset, confidential by default	Consumer P2P, not a merchant system
Daimo, Rozo, Sequence Trails	Intent-based any-token checkout	Mostly EVM + Solana; payment only
BitPay, CoinGate, NOWPayments	Invoice + deposit-address checkout	The source of today's pain points
Stripe, Shopify (USDC)	Stablecoins settled to fiat	Few tokens and chains; centralized
Hackathon widgets	1Click wrappers	Same ceiling, built in a weekend

> **Differentiation:** everyone above stops at "payment received." Morva starts there.

7. Solution

Morva is built in four layers. Layer 1 is table stakes; layers 2–4 are the moat.

Layer 4 · Privacy + agents
Confidential mode · x402 / MPP agent payments

Layer 3 · Ops agent
Shade Agent per merchant: refunds by policy, reconciliation, sweeps

Layer 2 · Commerce
Orders · hold / capture / void · refund to source · splits · webhooks · ledger

Layer 1 · Pay
Wallet, passkey and deposit paths · unified balance across chains · exact-output quotes

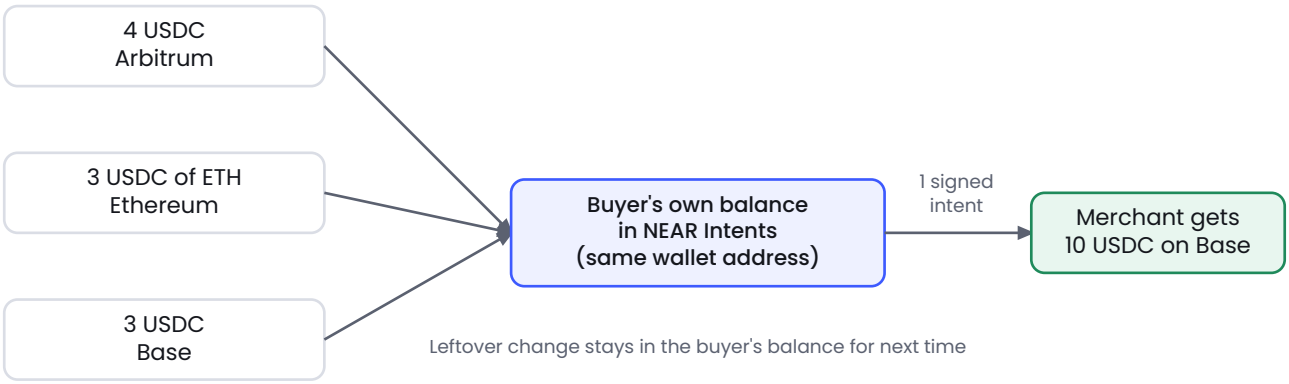
Foundation · NEAR Intents
Verifier ledger (intents.near) · 1Click API · market makers · 35 chains

Layer 1 · Pay

A universal checkout with three ways to pay. The **wallet path**: connect, see one total, sign once. The **signed-intent path**: buyers whose funds already sit in NEAR Intents sign one intent, with no on-chain deposit to wait for; this also supports passkeys (WebAuthn). The **deposit path**: get a one-time address and QR code and pay from any wallet or exchange, with no connecting or signing in Morva's interface. All use exact-output quotes, so the merchant receives the exact invoice amount.

Unified balance: pay from several chains at once

On NEAR Intents, a wallet's address is its account. Funds the buyer deposits from Arbitrum, Base or Ethereum land in **one balance** controlled by that same wallet. Morva detects the buyer's funds across supported chains, gathers the ones worth moving into the buyer's own balance, and the buyer pays with **one signed intent**.



- **Nothing gets stuck.** If one leg fails, the other funds are safe in the buyer's own balance, not half-delivered to an unpaid order.
- **Change stays with the buyer.** Any surplus remains in their balance, ready for the next purchase.
- **The second payment is one signature.** After the first gather, the buyer pays from a single balance with no gas and no chain switching.

- **Limits.** The first gather needs one approval per source chain (gasless for USDC via a relayer; native ETH still needs gas). Dust that costs more to move than it is worth is skipped. Chains NEAR Intents doesn't support (e.g. Arc and Celo today) are excluded.

Layer 2 · Commerce

Merchant funds settle into the merchant's own **NEAR Intents balance**, so settlement is a ledger update and the merchant withdraws to any chain when they choose. On top of that: orders and receipts; **hold → capture → void → refund** through an escrow contract on NEAR (the Verifier has no native hold, so Morva provides it); **refund to source**, where the merchant refunds in USDC and the buyer receives their original token on their original chain; partial refunds; **revenue splits** to multiple recipients; merchant webhooks; and an exportable ledger.

Layer 3 · Ops agent

Each merchant gets a Shade Agent running in a trusted execution environment. It enforces the merchant's policies automatically: refund on cancellation, capture on check-in, sweep balances to treasury, reconcile payments against orders and flag anomalies. It is non-custodial and verifiable: the agent can only do what the merchant's on-chain policy allows.

Layer 4 · Privacy and agents

Confidential mode keeps a merchant's revenue off public explorers, with selective disclosure for accountants or regulators. **Agent-ready** checkout accepts payments from AI agents through x402 and MPP using the same integration.

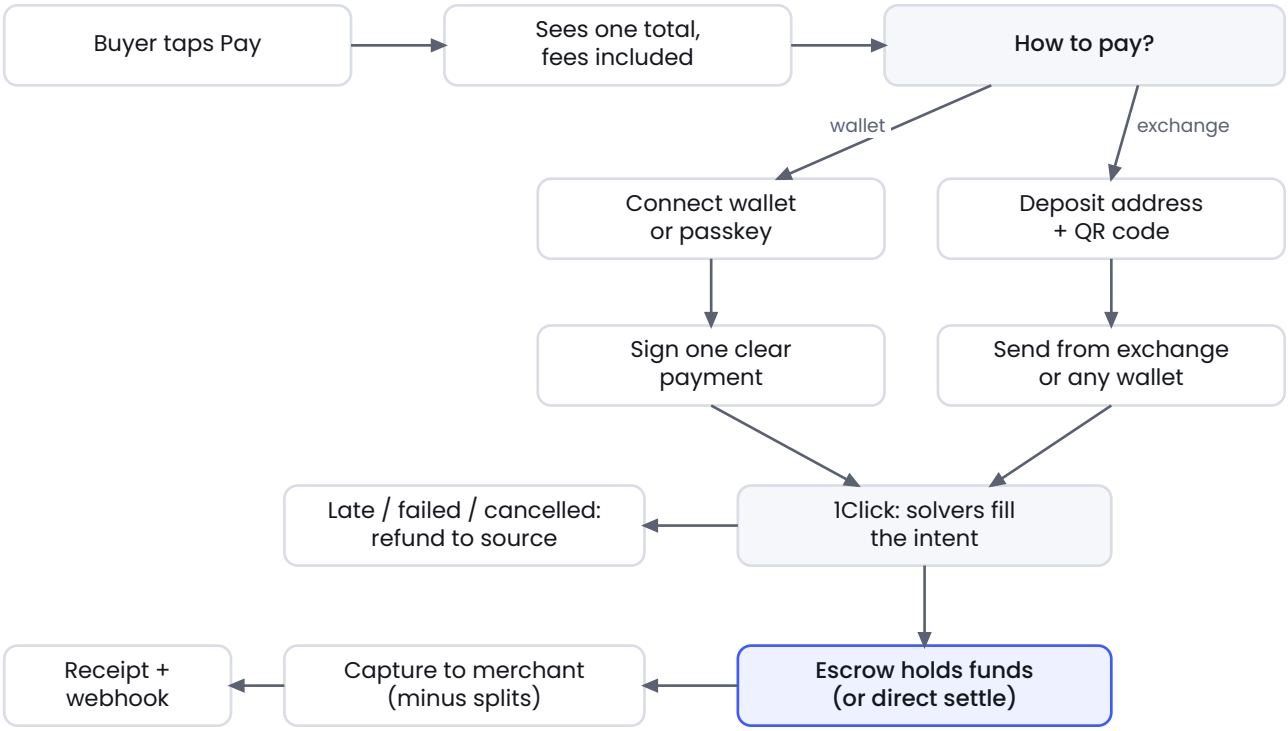
> **Signature demo:** an organizer cancels an event. Every attendee is refunded automatically, each in the token and on the chain they paid from. Nobody files a ticket.

8. Features and priorities

Feature	Priority	Hackathon
Hosted checkout page and payment links (QR, shareable)	P0	Yes
Wallet path and deposit-address path	P0	Yes
Signed-intent path (incl. passkeys via WebAuthn)	P0	Yes
Merchant balances in NEAR Intents; withdraw to any chain	P0	Yes
Unified balance: pay from several chains in one checkout	P1	Yes
Gasless USDC gather via permit signatures and a relayer	P2	—
Exact-output quotes; one all-in total	P0	Yes
Order model, receipts, merchant webhooks	P0	Yes
Merchant dashboard: orders, payouts, settings	P0	Yes
Escrow: hold, capture, void	P0	Yes
Refund to source (full and partial)	P0	Yes
Automatic handling of late, short and over payments	P0	Yes
Revenue splits	P1	Simple version
Ops agent with one policy (auto-refund on cancel)	P1	Yes
Confidential mode toggle	P1	Yes
Embeddable widget and React component	P1	Partial
Ledger CSV export	P1	Yes
Event ticketing integration (PartyStack)	P1	Yes
More agent policies: capture on check-in, sweeps, reconciliation	P2	—
x402 / MPP agent payments	P2	—
Subscriptions	P2	—
Shopify app and WooCommerce plugin	P2	—
Local-currency settlement	P3	—

9. User flows

Both buyer paths end the same way: solvers deliver the merchant's chosen asset, and anything late, failed or cancelled is refunded to source automatically.



9.1 Buyer: wallet path

1. Opens the payment link or clicks "Buy ticket."
2. Sees the item, price in USD, and **one total with fees included**, before any wallet prompt.
3. Taps **Pay** and connects a wallet (or creates a passkey wallet).
4. Checkout reads balances across chains and preselects the best token and chain; the buyer can change it.
5. Signs **one** plain-language prompt, e.g. "Pay 25.00 USDC to Lagos Builders Night."
6. Sees live status: *Sent* → *Confirming* → *Paid*.
7. Receives a receipt (and ticket) on screen and by email.

9.2 Buyer: deposit path (no wallet connection)

1. Taps **Pay from an exchange or another wallet**.
2. Picks the asset they will send, e.g. USDT on Tron, BTC or SOL.
3. Gets a one-time deposit address, QR code, exact amount, and a countdown sized to that chain (longer for Bitcoin).
4. Enters a **refund address**. This is required for exchange payers, since exchange addresses may not credit returned funds.
5. Sends funds, watches live status, and receives a receipt.

9.3 Buyer: paying from several chains (unified balance)

1. Connects a wallet; checkout shows one number, e.g. "$10.40 available across 3 chains," with a per-chain breakdown.

2. Morva plans the payment: picks sources, skips dust, and marks unsupported chains.
3. If funds fall short, checkout says so up front, e.g. "$3.00 more needed," rather than failing halfway.
4. Buyer approves one transfer per source chain into their own NEAR Intents balance.
5. Buyer signs **one** intent to pay the merchant; any change stays in their balance.

9.4 Merchant onboarding

1. Signs up with email or a wallet.
2. Picks a **settlement asset and chain** (default: USDC held in their NEAR Intents balance) and an optional automatic payout address.
3. Sets defaults: hold payments on/off, refund policy, confidential mode on/off, split recipients.
4. Creates a product, ticket type or payment link, and gets a URL and QR code, or API keys and a webhook endpoint.
5. Uses the dashboard to see orders, holds, refunds, payouts and exceptions.

9.5 Hold and capture (events)

1. Buyer pays; funds go to **escrow** and the order becomes `held`.
2. On check-in or after the event (organizer action or agent policy), the order is `captured` and funds go to the organizer, minus splits.
3. If the event is cancelled, the order is `voided` and refunded to source automatically.

9.6 Refund to source

1. Merchant clicks **Refund** (full or partial), or the ops agent triggers it by policy.
2. Morva requests a 1Click quote from the merchant's settlement asset to the buyer's **original token and chain**.
3. Funds route through NEAR Intents to the buyer's original address, or to the refund address they provided.
4. Buyer receives a notice such as "You've been refunded 25.00 USD in ETH on Arbitrum."

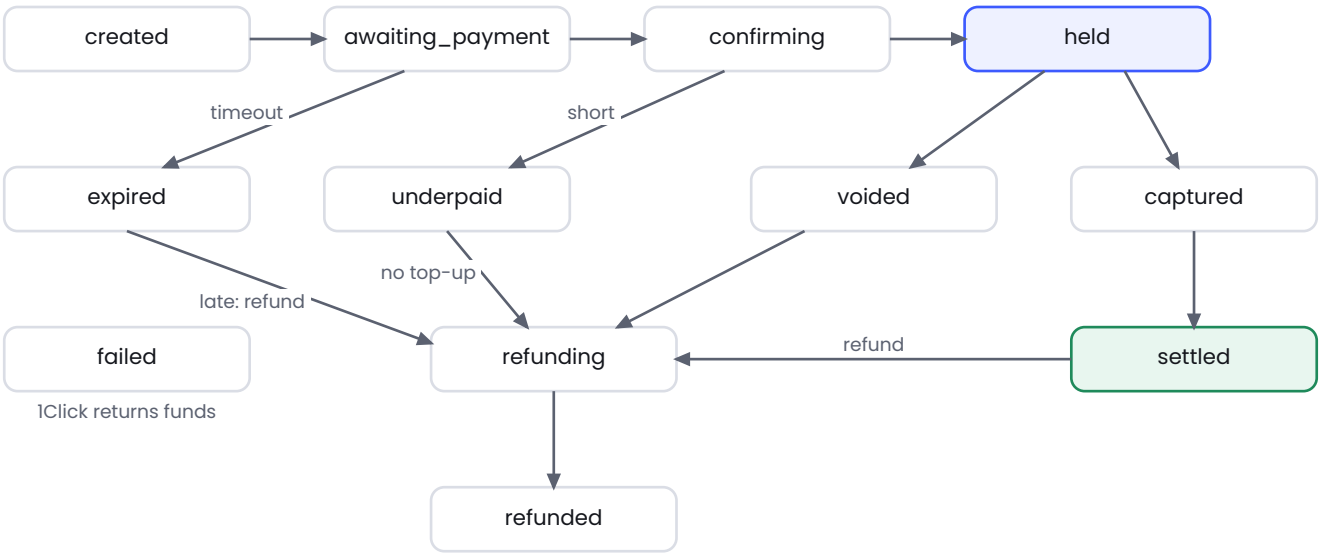
9.7 Failure handling

Every error message states what happened and what happens next, for example: "Your payment arrived late. We've refunded 25.00 USDC to your wallet."

Case	Morva behavior
Paid short, within tolerance	Accept and mark paid
Paid short, outside tolerance	Offer a one-tap top-up on the same order; otherwise auto-refund
Paid too much	Order completes; excess refunded automatically
Paid after the window	Re-quote; complete if the price moved within tolerance, otherwise auto-refund in full
Swap failed	1Click returns funds to the refund address; order marked failed and buyer notified
Buyer closed the tab	Payment link stays valid; order marked paid by webhook when funds land
Merchant cancels	Void the hold before capture, or one-click refund after
One leg of a multi-chain gather fails	Other legs stay in the buyer's own balance; buyer retries or adds another source
Buyer's total across chains is short	Shown before any approval, with the amount still needed

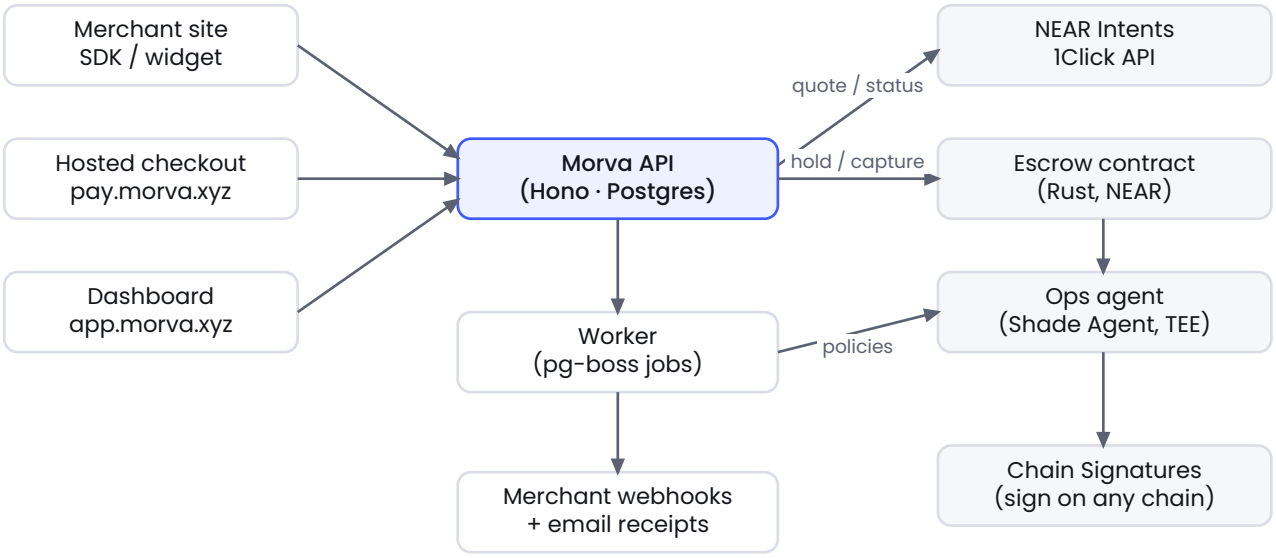
10. Order lifecycle

Every order follows one state machine. Orders without a hold move straight from `confirming` to `settled`.



State	Meaning
created	Order exists; no quote yet
awaiting_payment	Quote issued; deposit address active
confirming	Deposit detected; 1Click swap in progress
held	Settlement asset sits in escrow
captured / settled	Funds released to merchant and split recipients
voided	Hold cancelled before capture
underpaid / expired	Short or late payment awaiting top-up, re-quote or refund
refunding / refunded	Refund-to-source intent in flight / complete (also partially_refunded)
failed	Swap failed; 1Click returned funds to the refund address

11. System architecture



Component	Role	Technology
Checkout (apps/checkout)	Hosted pay pages, payment links, iframe embed	Next.js, Tailwind
Dashboard (apps/app)	Merchant console and marketing site	Next.js, Tailwind
API (apps/api)	Orders, quotes, webhooks, ledger, merchant auth	Node, Hono, Postgres
Worker	Status polling, retries, refunds, payouts	Node, pg-boss
Intents client (packages/intents)	1Click quotes, deposits, status, confidential flag, appFees	TypeScript
Escrow contract	Hold, capture, void, refund, splits	Rust, near-sdk
Ops agent	Policy enforcement, reconciliation	Shade Agent framework (TEE)
SDK and widget	Embeds for merchant sites	TS SDK, React, web component

Payment data flow

```
Buyer    -> Checkout   : open link / click Pay
Checkout -> API        : create order
API      -> 1Click     : EXACT_OUTPUT quote (recipient = escrow or merchant,
                         refundTo, appFees, confidential?)
1Click   -> API        : one-time deposit address + amount + deadline
Buyer    -> origin chain: send funds (wallet signature or manual deposit)
Worker  <-> 1Click     : poll status  PENDING -> SUCCESS | REFUNDED | FAILED
Solvers  -> NEAR       : deliver settlement asset to escrow (held) or merchant
API      -> Merchant   : webhook order.held / order.paid
Merchant / ops agent   : capture | void | refund  ->  escrow contract
Refund   -> 1Click     : quote settlement asset -> buyer's original token + chain

Unified balance variant
Buyer    -> each chain : deposit into own NEAR Intents balance (one per source)
Buyer    -> 1Click     : one signed intent (depositType INTENTS) -> merchant
```

12. NEAR integration

How NEAR Intents works

A buyer states the outcome they want; market makers compete to fill it; a smart contract on NEAR settles it atomically, so it either completes or refunds. Three layers sit underneath:

Layer	What it is	Why it matters to Morva
Verifier contract (intents.near)	A ledger holding token balances for every account	Buyer and merchant balances live here; moving them is instant
Message bus	Off-chain network where market makers quote and fill	All routing and liquidity, without Morva touching it
Bridges	Move tokens between NEAR and external chains	Buyers pay from Bitcoin, Tron, Solana and EVM chains

Accounts are derived from a wallet's public key with no registration. Intents are small signed instructions: `transfer`, `token_diff`, `ft_withdraw` and `add_public_key`. Supported signing standards include erc191, nep413, raw_ed25519, webauthn, ton_connect, sep53 and tip191.

How Morva uses it

NEAR component	How Morva uses it
1Click Swap API	Quotes, one-time deposit addresses, swap status, automatic refunds on failed swaps. Exact-output quotes so the merchant receives the invoice amount. Partner key kept server-side to avoid the 0.25% fee on unauthenticated quotes.
Swap types	EXACT_OUTPUT for invoices (short deposits refunded, excess refunded). ANY_INPUT is not used for checkout: no fixed rate, $1,000 sweep threshold, no refunds, partners only.
Signed intents (depositType INTENTS)	One-signature payments from balances already in NEAR Intents, including passkeys; also used for unified balance and refunds.
Verifier balances	Merchant settlement lands in the merchant's NEAR Intents balance; splits are a batch of transfer intents.
simulate_intents, quote signatures	Dry-run refunds before sending; prove to buyers that a quote really came from 1Click.
Explorer and Account APIs	Transaction history and balances for the dashboard and reconciliation.
appFees	Morva's per-payment fee, collected on every swap (default 50/50 share with the protocol).
Escrow contract (Morva)	The Verifier has no native hold primitive. Morva's Rust contract holds settled USDC; on capture it deposits into the merchant's NEAR Intents balance via ft_transfer_call to intents.near; on cancel it routes funds to refund to source.
Chain Signatures	Lets Morva's contract and agent sign refund transactions on the buyer's original chain.
Shade Agents	Per-merchant ops agent in a TEE, bound to an agent contract that enforces the merchant's policy.
Confidential Intents	Merchant opt-in privacy, one flag on 1Click requests, with selective disclosure.
MPP / x402 (later)	Accept AI-agent payments through the same checkout and settlement.

Integration rules

- Always set `refundTo` to an address Morva or the buyer controls. Some swap types retry every five minutes instead of refunding.
- Size payment windows per origin chain: minutes for fast chains, 45–60 minutes for Bitcoin.
- Warn exchange payers that exchange deposit addresses may not credit returned funds; require a personal refund address.
- Disclose the trust model: 1Click temporarily transfers assets to a trusted swapping agent during execution.
- Cross-contract calls from the Verifier may not finish in order: confirm each step (e.g. a withdrawal) before submitting a dependent one.
- Only plan unified-balance legs on supported chains, and skip legs whose cost exceeds their value.

13. Data model

Entity	Key fields
merchant	id, name, intents_account, settlement_asset, settlement_chain, payout_address, confidential_default, policies
product / ticket_type	id, merchant_id, name, price_usd, inventory, hold_by_default
payment_link	id, merchant_id, product_id, amount_usd, expires_at, metadata
order	id, merchant_id, status, amount_usd, settlement_amount, hold, buyer_email, created_at
payment	id, order_id, origin_asset, origin_chain, origin_address, deposit_address, quote_id, tx_hashes, status
gather_leg	id, order_id, chain, asset, amount, intents_account, tx_hash, status
refund	id, order_id, amount, destination_asset, destination_chain, destination_address, quote_id, status
split	id, merchant_id, recipient, share_bps
ledger_entry	id, order_id, type (capture, refund, fee, split), asset, amount, tx_hash, created_at
webhook_endpoint / event	url, secret, event_type, payload, delivery_attempts, status

14. API surface

Endpoint	Purpose
POST /v1/orders	Create an order (amount, product, hold, metadata)
POST /v1/orders/:id/quote	Get a quote for a chosen origin asset and chain
POST /v1/orders/:id/plan	Plan a multi-chain payment from the buyer's balances (sources, skipped dust, shortfall)
GET /v1/orders/:id	Order status and payment details
POST /v1/orders/:id/capture	Release held funds to the merchant and splits
POST /v1/orders/:id/void	Cancel a hold and refund to source
POST /v1/orders/:id/refunds	Full or partial refund to source
POST /v1/payment-links	Create a shareable link and QR code
GET /v1/ledger	Export ledger entries (JSON or CSV)
Webhooks	order.created, order.paid, order.held, order.captured, order.refunded, order.failed

Webhooks are signed with an HMAC secret per endpoint and retried with exponential backoff.

15. Repository and tech stack

Organization

Repository	Contents
morva	Main monorepo: apps, packages, contracts, ops agent
morva-shopify	Shopify app (later; uses Shopify's CLI and review process)
morva-woocommerce	WooCommerce plugin in PHP (later; released via WordPress.org)
morva-examples	Public integration examples

Monorepo layout (pnpm workspaces + Turborepo)

```
morva/
  apps/
    app/          Next.js: marketing site + merchant dashboard
    checkout/     Next.js: hosted pay pages (pay.morva.xyz/[id])
    docs/         documentation site
    api/          Node (Hono): orders, webhooks, ledger, 1Click calls
      worker/     status polling, retries, refunds, sweeps
  packages/
    core/         order model, types, payment state machine
    intents/      NEAR Intents client: quotes, deposits, status, confidential
    sdk/          framework-agnostic JS SDK
    react/        React hooks + <Checkout/> component
    widget/       web component for any site
    ui/           shared components + Tailwind preset
    config/       tsconfig, eslint, tailwind base
  contracts/
    escrow/       Rust (near-sdk): hold, capture, void, refund, splits
  agents/
    ops-agent/    Shade Agent: refund policies, reconciliation, sweeps
```

Why these choices

- **Next.js for hosted apps.** Keeps the 1Click partner JWT server-side, renders link preview cards for payment links shared on WhatsApp and X, and loads checkout fast on mobile data.
- **Plain JS / React bundles for merchant installs.** Merchants on Shopify, WordPress, Vite or plain HTML can drop Morva in without using Next.js.
- **Separate Node service for the backend.** Payments need long-running jobs (polling, Bitcoin windows, retries, refunds) that serverless functions can't run reliably. Postgres stores orders and the ledger; pg-boss runs jobs on the same database.
- **Rust for the escrow contract.** It holds customer funds, so it uses NEAR's primary, best-supported contract language.
- **Separate subdomains.** pay.morva.xyz for checkout and app.morva.xyz for the dashboard keep checkout security headers strict and allow safe iframe embedding.

16. Security and trust

- **Non-custodial by design.** Morva's servers never hold funds. Funds sit in the escrow contract, and only contract rules release them.

- **Buyer funds stay the buyer's.** Unified-balance gathers move funds into the buyer's own NEAR Intents balance, never into a Morva account.
- **Plain-language signing.** One prompt per payment that states the amount and recipient. No unlimited token approvals.
- **Secrets server-side.** The 1Click JWT and webhook secrets never reach the browser.
- **Agent guardrails.** The ops agent runs verified code in a TEE and can only perform actions the merchant's on-chain policy allows.
- **Escrow audit** before mainnet volume beyond pilot limits; per-order and per-day caps during the pilot.
- **Compliance hooks.** Wallet screening on payments, selective disclosure in confidential mode, and full ledger export.

17. Business model

Stream	Detail
Transaction fee	Fee per payment via 1Click appFees, e.g. 0.5–1%, well below 2–4% card fees
Pro plan	Monthly subscription for ops-agent policies, confidential mode, splits and advanced exports
Enterprise / events	Custom pricing for large organizers and platforms

Exact fee levels are to be validated with pilot merchants.

18. Success metrics

Metric	Hackathon target	Why it matters
Checkout completion (quote → paid)	≥ 70%	Core conversion; Web3 baseline is far lower
Median time to paid	< 60 seconds (non-BTC)	Speed is the felt experience
Payments needing manual support	0	Proves self-resolving failure handling
Refund-to-source success rate	100% of attempted	Proves the commerce layer
Multi-chain payments completed	≥ 5	Proves the unified balance
Live merchants	≥ 1 (PartyStack event)	Real volume, not a demo
Live payments	≥ 25 real buyers	Evidence for judges and NEAR

19. Risks and mitigations

Risk	Mitigation
NEAR ships merchant checkout itself (Universal Send is close)	Compete on the commerce layer and events wedge, not on a basic pay button
HOT Pay is the ecosystem incumbent	Don't compete on token count; win on holds, refunds, splits and ops
1Click trust model (temporary custody by a swapping agent)	Disclose clearly; keep escrow non-custodial; monitor NEAR's roadmap
Escrow contract bug	Small, audited contract; pilot caps; staged rollout
Exchange deposits not credited on refund	Require a personal refund address on the deposit path
Many copycat checkouts after the public idea	Ship the hard parts first; get real merchant volume early
Buyer funds on chains NEAR Intents doesn't support (e.g. Arc, Celo)	Exclude them clearly in the plan; track the chain list as it grows
Moving small balances costs more than they are worth	Skip dust automatically and explain why
First multi-chain gather needs several approvals	Gasless USDC via relayer; passkey wallets; one signature on every later payment
Regulatory requirements per market	Start with crypto-native merchants; defer fiat settlement and local payouts

20. Roadmap

Phase	Scope
Week 1	Spike to verify open questions 7-10; monorepo scaffold; 1Click integration; exact-output checkout with wallet, signed-intent and deposit paths; order model and webhooks
Week 2	Escrow contract on NEAR testnet: hold, capture, void; refund to source; unified-balance gather and one-intent payment
Week 3	Ops agent with one policy (auto-refund on cancel); confidential-mode toggle; dashboard
Week 4	Mainnet pilot: real ticket sales for one PartyStack event; demo video; submission
After hackathon	More agent policies, splits UI, React and widget packages, escrow audit
Next	Shopify and WooCommerce plugins, x402 / MPP agent payments, subscriptions
Later	Local-currency settlement for markets such as Nigeria

21. Open questions

1. Should the escrow hold settled funds on NEAR only, or also support holds on the merchant's chosen chain?
2. What underpayment tolerance should be the default (e.g. 0.5% or $0.50)?
3. What fee level do pilot merchants accept: flat percentage, or percentage plus Pro plan?
4. Should buyers without a wallet get a passkey wallet in v1, or rely on the deposit path?

5. How are disputes handled when a buyer and merchant disagree about a refund?
6. What compliance checks apply to the pilot markets and order sizes?
7. To verify: when 1Click delivers USDC to the escrow contract, is the contract notified with an order reference, or only a plain transfer?
8. To verify: can a contract deposit into intents.near on behalf of the merchant by naming them in msg?
9. To verify: does a buyer's 0x address map to the same NEAR Intents account across Arbitrum, Base and Ethereum, as the docs describe?
10. To verify: which supported chains allow gasless USDC permits for the relayer, and how do wallets handle the network switch?

22. Sources

NEAR Intents – 1Click API — https://docs.near-intents.org/near-intents/1click-api

NEAR Intents – distribution channels — https://docs.near-intents.org/integration/distribution-channels

NEAR Intents – swap types — https://docs.near-intents.org/integration/distribution-channels/1click-api/swap-types

NEAR Intents – fee configuration — https://docs.near-intents.org/integration/distribution-channels/1click-api/fee-config.md

NEAR Intents – what are intents — https://docs.near-intents.org/getting-started/what-are-intents

NEAR Intents – intent types and execution — https://docs.near-intents.org/integration/verifier-contract/intent-types-and-execution.md

NEAR Intents – account abstraction — https://docs.near-intents.org/integration/verifier-contract/account-abstraction.md

NEAR Intents – deposits — https://docs.near-intents.org/integration/verifier-contract/deposits-and-withdrawals/deposits.md

NEAR Intents – signed intent execution — https://docs.near-intents.org/integration/distribution-channels/1click-api/quickstart/signed-intent-execution.md

NEAR Intents – chain support — https://docs.near-intents.org/resources/chain-support.md

MPP – NEAR Intents charge — https://mpp.dev/payment-methods/nearintents/charge

NEAR docs – Shade Agents — https://docs.near.org/ai/shade-agents/concepts/framework-overview

Glitchwire – Confidential Intents GA — https://glitchwire.com/news/nears-confidential-intents-reaches-general-availability-bringing-privacy-infrast/

Datawallet – NEAR statistics 2026 — https://www.datawallet.com/crypto/near-protocol-statistics

HOT Pay — https://hot-labs.org/pay/

Bankless – NEAR Universal Send — https://www.bankless.com/fr/read/news/near-unveils-universal-send-for-confidential-crosschain-payments

Shopify Engineering – Commerce Payments Protocol — https://shopify.engineering/commerce-payments-protocol

SolCard – crypto ownership — https://www.solcard.cc/blog/how-many-people-own-crypto

Ecommerce Fastlane – crypto payments — https://ecommercefastlane.com/crypto-payments-for-ecommerce/

Coinbound – Web3 funnel metrics — https://coinbound.io/web3-marketing-metrics/

UserTourKit – Web3 onboarding — https://usertourkit.com/blog/web3-app-onboarding

CoinGate – crypto payment errors — https://coingate.com/blog/post/what-goes-wrong-crypto-payments

ForkLog – 2025 crypto scams — https://forklog.com/en/crypto-scammers-stole-a-record-14bn-in-2025/

CryptumPay – merchant survey figures —
https://www.cryptumpay.com/blog-posts/how-to-accept-cryptocurrency-payments-online-store